

Transferencia del Aprendizaje con Deep Network Designer

Autor: Dr. Aboud Barsekh Onji

Institución: Universidad Anáhuac México

Contacto: aboud.barsekh@anahuac.mx

ORCID: 0009-0004-5440-8092

Fuente: MathWorks - Transfer Learning with Deep Network Designer

Introducción

La **transferencia del aprendizaje** (*transfer learning*) es una de las técnicas más poderosas y prácticas del deep learning moderno. En lugar de entrenar una red neuronal profunda desde cero –lo que requiere millones de imágenes y días de cómputo– se toma una red ya entrenada en un problema grande (como clasificar 1000 categorías de ImageNet) y se adapta para resolver un problema nuevo con mucho menos datos y tiempo.

La intuición es sencilla: las primeras capas de una CNN ya han aprendido a detectar bordes, texturas y formas básicas; esas representaciones son útiles para casi cualquier tarea visual. Solo es necesario reentrenar las últimas capas para que el modelo “aprenda” las clases nuevas.

Este ejemplo utiliza la herramienta visual **Deep Network Designer** de MATLAB para modificar la red **SqueezeNet** y clasificar 5 categorías de artículos promocionales de MathWorks (gorra, cubo, naipes, destornillador y linterna).

1. Preparación del Conjunto de Datos

1.1 Estructura de los datos

El conjunto de datos *MathWorks Merch* contiene **75 imágenes** organizadas en carpetas según su clase:

```
MerchData/  
  +--- cap/  
  +--- cube/  
  +--- playing cards/  
  +--- screwdriver/  
  +--- torch/
```

Esta organización en subcarpetas es el estándar de MATLAB para `imageDatastore`, que infiere automáticamente la etiqueta de cada imagen a partir del nombre de la carpeta.

1.2 Carga con imageDatastore

```
folderName = "MerchData";
unzip("MerchData.zip", folderName);

imds = imageDatastore(folderName, ...
    IncludeSubfolders=true, ...
    LabelSource="foldernames");
```

Un `imageDatastore` es un contenedor eficiente que lee imágenes en lotes durante el entrenamiento, lo que evita cargar todo el dataset en RAM. Es especialmente importante cuando los datasets son grandes.

1.3 División del dataset

Se divide el conjunto en tres partes:

Partición	Proporción	Uso
Entrenamiento	70%	Ajustar los pesos de la red
Validación	15%	Monitorizar sobreajuste durante el entrenamiento
Prueba	15%	Evaluar el rendimiento final (nunca visto por el modelo)

```
[imdsTrain, imdsValidation, imdsTest] = ...
    splitEachLabel(imds, 0.7, 0.15, 0.15, "randomized");
```

¿Por qué separar validación de prueba?

La validación se usa para ajustar hiperparámetros (p. ej., tasa de aprendizaje, épocas). Si usáramos el set de prueba para esto, estaríamos “filtrando” información del conjunto de prueba al proceso de diseño, lo que daría métricas sobreoptimistas.

2. Selección de la Red Preentrenada: SqueezeNet

2.1 ¿Qué es SqueezeNet?

SqueezeNet es una CNN publicada en 2016 diseñada para ser muy compacta (~1.24 millones de parámetros, comparado con los ~60 M de AlexNet), manteniendo precisión comparable en ImageNet. Su arquitectura se basa en módulos llamados *Fire modules*, compuestos de capas *squeeze* (convoluciones 1x1) y *expand* (convoluciones 1x1 y 3x3).

Ventajas para este ejemplo: - No requiere paquete de soporte adicional en MATLAB - Tamaño de entrada: **227 x 227 x 3** píxeles (RGB) - Entrenada en ImageNet (1000 clases) - Rápida de reentrenar en CPU

```
inputSize = [227 227 3];
```

2.2 Abrir Deep Network Designer

deepNetworkDesigner

Desde la interfaz, se selecciona SqueezeNet de la galería de redes preentrenadas. La app muestra una vista alejada de la arquitectura completa en el panel **Designer**, donde se puede navegar e inspeccionar cada capa individualmente.

Tip: Para explorar la red en la app, usa **Ctrl + rueda del ratón** para hacer zoom y las teclas de flecha para navegar. Al seleccionar una capa, el panel **Properties** muestra sus parámetros.

3. Modificación de la Red para Transferencia del Aprendizaje

3.1 ¿Qué hay que cambiar?

La última capa convolucional de SqueezeNet (**conv10**) tiene **1000 filtros** (uno por clase de ImageNet). Como nuestro problema tiene solo **5 clases**, debemos:

1. **Desbloquear** la capa **conv10**
2. Cambiar **NumFilters** de 1000 -> **5**
3. Aumentar las **tasas de aprendizaje** de esa capa para que aprenda rápido en las clases nuevas

3.2 Factores de tasa de aprendizaje

Parámetro	Valor	Interpretación
WeightLearnRateFactor	10	Los pesos de esta capa se actualizan 10x más rápido que el resto
BiasLearnRateFactor	10	Los sesgos también aprenden más rápido

Esto es esencial porque las capas iniciales ya tienen buenos filtros generales; solo la última capa necesita adaptarse agresivamente a las nuevas clases. En el panel **Properties** de la app se verán los campos **NumFilters**, **WeightLearnRateFactor** y **BiasLearnRateFactor** listos para editar tras desbloquear la capa.

3.3 Verificación con Deep Learning Network Analyzer

Después de modificar la red, se hace clic en **Analyze**. Si no hay errores ni advertencias, la red está lista. El analizador mostrará un resumen con el número total de capas, parámetros y el tamaño del tensor de salida, confirmando que

la red espera una entrada de 227x227x3 y produce 5 clases de salida. Luego se exporta con **Export** -> queda guardada en la variable **net_1**.

Nota importante (versiones anteriores a R2023b): En versiones previas no se puede desbloquear una capa directamente. En esos casos había que reemplazar `conv10` con una nueva capa convolucional con `FilterSize = [1 1]` y `NumFilters = 5`.

4. Aumento de Datos (*Data Augmentation*)

Con solo 75 imágenes, el riesgo de **sobreajuste** es muy alto. El aumento de datos genera variantes artificiales de las imágenes de entrenamiento aplicando transformaciones aleatorias.

4.1 Transformaciones aplicadas

Transformación	Parámetro	Efecto
Volteo horizontal aleatorio	<code>RandXReflection=true</code>	Simula imágenes en espejo
Traslación horizontal aleatoria	<code>RandXTranslation=[-30 30]</code>	Desplaza hasta 30 px en X
Traslación vertical aleatoria	<code>RandYTranslation=[-30 30]</code>	Desplaza hasta 30 px en Y

```
pixelRange = [-30 30];
```

```
imageAugmenter = imageDataAugmenter( ...  
    RandXReflection=true, ...  
    RandXTranslation=pixelRange, ...  
    RandYTranslation=pixelRange);
```

```
augimdsTrain = augmentedImageDatastore(inputSize(1:2), imdsTrain, ...  
    DataAugmentation=imageAugmenter);
```

Para validación y prueba **no se aplica aumento** (solo se redimensiona):

```
augimdsValidation = augmentedImageDatastore(inputSize(1:2), imdsValidation);  
augimdsTest = augmentedImageDatastore(inputSize(1:2), imdsTest);
```

¿Por qué no aumentar validación/prueba? Queremos medir el rendimiento real del modelo sobre imágenes no modificadas. Aplicar aumento aleatorio introduciría varianza en las métricas de evaluación.

5. Opciones de Entrenamiento

```
options = trainingOptions("adam", ...  
    InitialLearnRate=0.0001, ...  
    MaxEpochs=8, ...  
    ValidationData=imdsValidation, ...  
    ValidationFrequency=5, ...  
    MiniBatchSize=11, ...  
    Plots="training-progress", ...  
    Metrics="accuracy", ...  
    Verbose=false);
```

5.1 Justificación de cada hiperparámetro

Opción	Valor	Justificación
Optimizador	adam	Adaptativo, converge bien con pocos datos
InitialLearnRate	0.0001	Bajo para no destruir los pesos preentrenados
MaxEpochs	8	En transfer learning converge rápido; muchas épocas llevan a sobreajuste
MiniBatchSize	11	Divide uniformemente las ~52 imágenes de entrenamiento (75x0.7~52; 52/11~4.7 iteraciones/época)
ValidationFrequency	5	Valida cada 5 iteraciones

5.2 El optimizador Adam

Adam (*Adaptive Moment Estimation*) combina las ideas de *momentum* y *RMSPProp*. Mantiene estimaciones de primer y segundo momento del gradiente:

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \theta_{t+1} &= \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t\end{aligned}$$

donde $\hat{m}_t$ y $\hat{v}_t$ son estimaciones corregidas por sesgo. Esto lo hace muy eficiente para datasets pequeños y heterogéneos.

6. Entrenamiento

```
net = trainnet(imdsTrain, net_1, "crossentropy", options);
```

La función `trainnet` usa la **pérdida de entropía cruzada** (*crossentropy*), adecuada para clasificación multiclase:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

donde y_i es la etiqueta verdadera (one-hot) y $\hat{y}_i$ la probabilidad predicha para la clase i .

Durante el entrenamiento, la app muestra la gráfica de progreso con la pérdida y la precisión en entrenamiento y validación. Una convergencia saludable mostrará ambas curvas de pérdida decreciendo juntas y sin gran brecha entre ellas.

7. Evaluación del Modelo

7.1 Clasificación del set de prueba

```
YTest = minibatchpredict(net, augimdsTest);
YTest = scores2label(YTest, classNames);
```

La función `minibatchpredict` procesa las imágenes en lotes para eficiencia. Devuelve puntuaciones (probabilidades) que luego se convierten a etiquetas con `scores2label`.

7.2 Matriz de confusión

```
TTest = imdsTest.Labels;
figure
confusionchart(TTest, YTest);
```

La **matriz de confusión** permite identificar exactamente qué clases se confunden entre sí. La diagonal principal muestra las predicciones correctas. Es mucho más informativa que la precisión global, especialmente cuando las clases están desbalanceadas. Por ejemplo, si el modelo confunde frecuentemente “screwdriver” con “torch”, la matriz lo revela de inmediato.

8. Predicción sobre una Imagen Nueva

```
im = imread("MerchDataTest.jpg");
im = imresize(im, inputSize(1:2));
X = single(im);

if canUseGPU
    X = gpuArray(X);
```

end

```
scores = predict(net, X);  
[label, score] = scores2label(scores, classNames);  
  
figure  
imshow(im)  
title(string(label) + " (Score: " + gather(score) + ")")
```

Nota: `single(im)` convierte la imagen a precision simple (float32), que es el tipo esperado por la red y mas eficiente en GPU. `gather(score)` devuelve el valor de GPU a CPU para mostrarlo. El titulo de la figura desplegara la clase predicha junto con su porcentaje de confianza.

9. Conceptos Clave – Tabla de Referencia

Concepto	Significado	Aplicacion en este ejemplo
Transfer Learning	Reutilizar pesos de una red entrenada en otra tarea	SqueezeNet preentrenada en ImageNet para clasificar 5 articulos
Fine-tuning	Reentrenar selectivamente las capas finales	Solo <code>conv10</code> se modifica agresivamente
imageDatastore	Contenedor eficiente para imagenes	Lee lotes sin cargar todo en RAM
Data Augmentation	Generar variantes artificiales del dataset	Volteos y traslaciones aleatorias
Adam	Optimizador adaptativo de gradiente	Converge rapido con pocos datos
Entropia cruzada	Funcion de perdida para clasificacion	Mide la distancia entre distribucion predicha y real
Matriz de confusion	Visualizacion de errores por clase	Identifica que clases se confunden
minibatchpredict	Prediccion eficiente por lotes	Clasifica multiples imagenes de prueba

scores2label	Convierte probabilidades en etiquetas	Obtiene la clase mas probable
---------------------	---	-------------------------------

10. Flujo Completo del Pipeline

```

Dataset (75 imágenes)
    |
    v
imageDatastore + splitEachLabel (70/15/15)
    |
    v
Deep Network Designer
  +-- Cargar SqueezeNet
  +-- Modificar conv10: NumFilters=5, LRF=10
  +-- Analizar + Exportar --> net_1
    |
    v
augmentedImageDatastore (resize + augmentation)
    |
    v
trainingOptions (Adam, lr=0.0001, 8 épocas)
    |
    v
trainnet(imdsTrain, net_1, "crossentropy", options)
    |
    v
minibatchpredict + confusionchart (evaluación)
    |
    v
predict (imagen nueva --> etiqueta + score)

```

11. Observaciones Pedagógicas y Extensiones

1. **¿Qué pasa si aumento el número de épocas?** La precisión de entrenamiento subirá, pero la de validación puede bajar (sobreajuste). Observa las curvas en la gráfica de progreso.
2. **¿Qué pasa si uso una tasa de aprendizaje más alta (p. ej., 0.01)?** Los pesos preentrenados de las capas iniciales se modificarán demasiado y se perderá el conocimiento transferido.

3. **¿Qué red usar si mi dataset es muy diferente a ImageNet?** Entrenar desde cero o usar técnicas como *domain adaptation*.
4. **¿Puedo usar otras redes preentrenadas?** Sí. GoogLeNet, ResNet-50, EfficientNet-b0, etc. Cada una tiene diferentes requisitos de tamaño de entrada y capacidad computacional.
5. **Experiment Manager:** Para buscar los mejores hiperparámetros de forma sistemática, MATLAB incluye la app *Experiment Manager* que ejecuta múltiples configuraciones en paralelo.